Project Detailed Design

For this phase of the project, students have the opportunity to demonstrate mastery of the following SLOs:

- SC2. Write appropriate interface and (if needed) implementation comments for a small component.

During each of the 2 sprints for this phase, students have the opportunity to demonstrate proficiency in the following SLOs:

- T1. Follow effective team communication practices.
    - Contribute to the creation of the Coding Standard Document or, review it, and communicate feedback to the author(s).
    - Complete the weekly status report in Teammates on time, correctly, and honestly.
- T4. Collaborate effectively in cooperative development/programming.
    - Attend the meeting, contribute, follow team contract requirements, etc. (PM: lead meeting)
- TE3. Identify configuration items and use a source code control tool in a small team-based project.
    - Create at least 2 PRs each week, except the LE and PM.
- SC1. Write appropriate unit tests for a small component (several functions, a single type, etc.).
- SC4. Select and use a defined coding standard in a small software project.
    - Contribute to the creation of the Coding Standard Document or, review it, and communicate feedback to the author(s).
- SC5. Write implementation documentation for tricky and non-obvious pieces of code.
    - This needs to be enforced by the LE.
- VV12. Automate testing in a small software project.
    - Again, this is enforced by the LE.
- VV13. Use a defect tracking tool to manage software defects in a small software project.
    - Uses Kanban board correctly.

For this phase, do the following:

- **Coding Standard Document (SC4)**
    - Upload it in your `docs/` repo with the name `CodingStandard.md`.
    - It does not have to be more than 1 or 2 pages.

- **Implementation (SC2)**
    - Create a server and design and implement the RESTful API. Implement tests to exercise the API. Mock the database -- hardcode data in the server instead of retrieving data from a database.
    - Update the client to retrieve data from the server, instead of using the hard-coded data. (Don't remove the logic/code that provided the hard-coded data -- just comment it out or isolate it so it isn't used by default.) The existing unit tests of the client should be unaltered, if possible, but may need to be extended when you implement additional user stories.

    - Implement 5 more user stories. Mock the data in the client first, get the client working, then add the REST API endpoints you need, and update the client to use it. Implement tests of these new API endpoints. Use Test-driven development if you can.
    - Push your upgraded client and server to your repo. Use the Git tagging mechanism to mark this prototype code version (the client/service repos) as `detailed_design`. (LE's responsibility)

- **Updated Materials**

    - Continue to update your project management files: UI Model, Domain Model, User Stories, SRS, etc.
    - Update your top-level README.md to include instructions on how to run your server and client, so that your merciful, kind professor can test your prototype.